

머신러닝 · 컴퓨터공학과 (4학년)

기말 프로젝트

CardioCare

종단간 머신러닝 시스템 구축

End-to-End Machine Learning System for Heart Disease Prediction

과목	기계학습
담당 교수	백우진
학번	202321853
이름	서우영

1. 문제 정의와 사용 목적

본 프로젝트의 주제는 심장병 관련 임상 데이터를 활용하여 환자의 심장병 가능성을 예측하는 머신러닝 모델을 구축하는 것이다. 사용한 데이터는 환자의 나이, 성별, 흉통 유형, 안정 혈압, 콜레스테롤 수치, 최대 심박수 등 여러 임상 정보를 포함하고 있으며, 최종적으로 target 값을 통해 심장병 여부를 예측하는 이진 분류 문제로 정의하였다.

이 프로젝트의 목적은 단순히 높은 성능의 모델을 만드는 것에 그치지 않는다. 실제 머신러닝 프로젝트의 전체 흐름을 반영하여 데이터 로딩, EDA, 전처리, 모델 학습, 모델 비교, 하이퍼파라미터 튜닝, MLflow 실험 관리, 추론, 테스트, Docker 패키징, GitHub Actions CI, 데이터 드리프트 모니터링까지 포함하는 End-to-End 머신러닝 파이프라인을 구축하는 것을 목표로 하였다.

본 프로젝트에서 만든 모델은 의료진의 최종 판단을 대체하기 위한 것이 아니라, 심장병 가능성을 예측하여 참고 정보를 제공하는 보조 도구로 사용되는 것을 전제로 한다. 따라서 모델의 예측 결과는 실제 진단이나 치료 결정을 대신할 수 없으며, 최종 의학적 판단은 반드시 전문 의료진이 수행해야 한다.

2. 데이터셋 및 EDA 핵심 결과

본 프로젝트에서는 Heart Disease Dataset을 사용하였다. 데이터셋은 총 1025개의 샘플과 14개의 컬럼으로 구성되어 있으며, target 컬럼은 심장병 여부를 나타낸다. target=0은 심장병 없음, target=1은 심장병 가능성 있음을 의미한다.

주요 입력 변수는 age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal로 구성된다. 이 변수들은 환자의 기본 정보, 혈압, 콜레스테롤, 심전도 결과, 최대 심박수, 운동 유발 협심증 여부 등 심장병 예측과 관련된 임상 정보를 포함한다.

EDA 과정에서는 head(), info(), describe()를 통해 데이터 구조를 먼저 확인하였다. 그 결과 모든 컬럼에서 결측값은 발견되지 않았으며, 데이터 타입도 대부분 정수형 또는 실수형으로 구성되어 있어 모델 학습에 바로 활용할 수 있는 형태였다. 또한 학습에 사용할 수 없는 빈 컬럼은 확인되지 않았다.

target 분포를 확인한 결과, target=1은 526개, target=0은 499개로 나타났다. 비율로 보면 target=1은 약 51.3%, target=0은 약 48.7%로 두 클래스가 비교적 균형적인 분포를 보였다. 따라서 극단적인 클래스 불균형 문제는 크지 않다고 판단하였다. 다만 의료 예측 문제에서는 단순 정확도보다 실제 심장병 환자를 놓치지 않는 것이 중요하므로, 이후 모델 평가에서는 accuracy만 사용하지 않고 balanced accuracy, recall, F1-score를 함께 사용하였다.



그림 1. Target Class Distribution: 심장병 여부(target)의 클래스 분포

연속형 변수인 age, trestbps, chol, thalach, oldpeak에 대해서는 boxplot을 생성하여 분포와 이상치를 확인하였다. 그 결과 chol, trestbps, oldpeak 등 일부 변수에서 이상치로 보이는 값이 확인되었다. 그러나 의료 데이터에서 극단값은 실제 고위험 환자의 특성을 반영할 수 있기 때문에 단순히 삭제하지 않고 유지하였다. 대신 이후 전처리 과정에서 연속형 변수의 스케일 차이를 줄이기 위해 StandardScaler를 적용하였다.

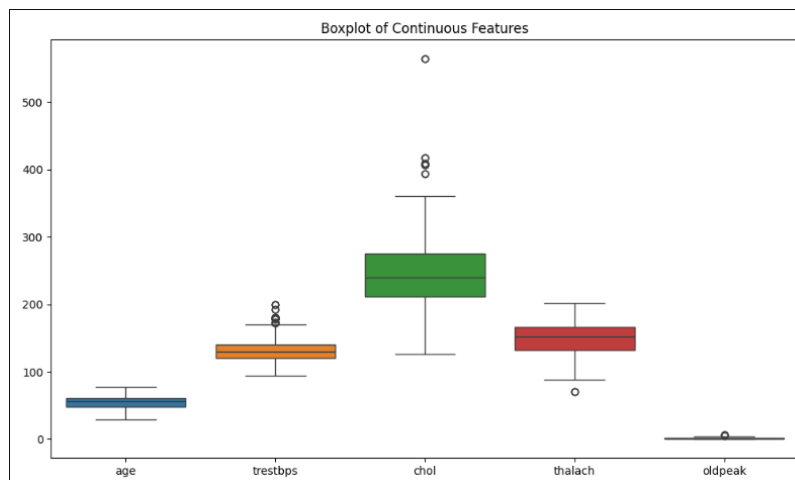


그림 2. Boxplot of Continuous Features: 주요 연속형 변수의 분포와 이상치 확인

EDA 과정에서 가장 중요한 발견은 중복 행이었다. 전체 1025개 행 중 723개의 중복 행이 확인되었다. 동일한 데이터가 학습 데이터와 테스트 데이터에 동시에 포함될 경우, 모델이 이미 본 데이터와 유사한 데이터를 평가하게 되어 성능이 과대평가될 수 있다. 따라서 본 프로젝트에서는 중복 행을 제거하기로 결정하였다. 중복 제거 후 최종 데이터는 302개 샘플로 구성되었다.

정리하면, EDA를 통해 target 분포는 비교적 균형적이고 결측값과 빈 컬럼은 없지만, 중복 행이 매우 많고 일부 연속형 변수에 이상치가 존재한다는 점을 확인하였다. 이에 따라 전처리 단계에서는 중복 제거, 연속형 변수 스케일링, 결측값 발생 가능성을 고려한

대치 과정을 포함한 재사용 가능한 전처리 파이프라인을 구성하였다.

3. 전처리 결정

전처리 과정에서는 먼저 중복 행을 제거하였다. 원본 데이터는 1025개 행이었으나, 723개의 중복 행을 제거한 후 302개의 고유 샘플이 남았다. 중복 제거는 모델 평가의 신뢰성을 높이기 위한 중요한 처리이다. 중복 데이터를 그대로 사용할 경우 학습 데이터와 테스트 데이터 사이에 동일하거나 매우 유사한 샘플이 포함될 수 있고, 이로 인해 실제보다 높은 성능이 나올 수 있기 때문이다.

그다음 입력 변수 X 와 정답 변수 y 를 분리하였다. `target` 컬럼은 예측 대상이므로 y 로 사용하고, 나머지 13개 컬럼은 입력 변수 X 로 사용하였다. 이후 전체 데이터를 학습 데이터와 테스트 데이터로 8:2 비율로 나누었다. 이때 `stratify=y` 옵션을 사용하여 학습 데이터와 테스트 데이터의 `target` 비율이 비슷하게 유지되도록 하였다.

X_train	241행, 13컬럼
X_test	61행, 13컬럼
y_train	241개
y_test	61개

중복 제거 후 데이터 분할 결과는 다음과 같다.

전처리 파이프라인에서는 연속형 변수와 범주형 변수를 구분하였다. 연속형 변수는 `age`, `trestbps`, `chol`, `thalach`, `oldpeak`로 설정하였다. 이 변수들은 값의 범위가 서로 다르기 때문에 `StandardScaler`를 적용하여 스케일을 맞추었다. 예를 들어 `chol`은 200대 값을 가지는 경우가 많고, `oldpeak`는 0~6 정도의 범위를 가지므로 모델에 따라 스케일 차이가 성능에 영향을 줄 수 있다.

범주형 변수는 `sex`, `cp`, `fbs`, `restecg`, `exang`, `slope`, `ca`, `thal`로 설정하였다. 이 변수들은 숫자로 표현되어 있지만 의미상 범주를 나타낸다. 결측값은 현재 데이터에는 존재하지 않았지만, 새로운 데이터가 들어올 가능성을 고려하여 파이프라인에는 결측값 처리 과정을 포함하였다. 연속형 변수는 중앙값으로 대체하고, 범주형 변수는 최빈값으로 대체하도록 구성하였다.

이상치의 경우 `chol`, `trestbps`, `oldpeak` 등 일부 연속형 변수에서 확인되었지만 단순 삭제하지 않았다. 의료 데이터에서 극단값은 실제 고위험 환자의 특성을 반영할 수 있기 때문이다. 따라서 이상치는 제거하지 않고 유지하되, 연속형 변수 스케일링을 통해 값의 범위 차이가 모델 학습에 과도하게 영향을 주지 않도록 처리하였다.

전처리 과정은 `ColumnTransformer`와 `Pipeline`을 사용하여 구성하였다. 이를 통해 연속형 변수와 범주형 변수에 서로 다른 전처리 방식을 적용하면서도, 학습·평가·추론 단계에서

동일한 흐름을 재사용할 수 있도록 하였다. 또한 학습 시점에는 학습 데이터에만 전처리 기준이 맞춰지고, 테스트 데이터에는 학습 데이터에서 학습된 기준으로 변환만 적용된다. 이 방식은 스케일링이나 결측값 대치 과정에서 테스트 데이터의 정보가 학습 과정에 섞이는 데이터 누수를 방지하는 데 중요하다.

4. 모델 학습 및 MLflow 실험 비교

모델 학습에 앞서 Random Forest 기반 SelectFromModel을 사용하여 특성 선택을 수행하였다. 특성 선택은 데이터 누수를 방지하기 위해 train/test split 이후 학습 데이터에 대해서만 수행하였다. 먼저 학습 데이터에 전처리 파이프라인을 적용한 뒤, Random Forest의 feature importance를 기준으로 median threshold 이상의 중요도를 가진 특성을 선택하였다.

특성 선택 결과 전체 13개 입력 변수 중 7개 특성이 선택되었다. 선택된 특성은 age, chol, thalach, oldpeak, cp, ca, thal이다. 특성 중요도 기준으로는 흉통 유형을 나타내는 cp가 가장 높게 나타났고, 최대 심박수인 thalach, 주요 혈관 수를 의미하는 ca, 운동 후 ST depression을 나타내는 oldpeak, thal 순으로 높은 중요도를 보였다.

순위	선택된 특성	중요도
1	cp	0.1814
2	thalach	0.1242
3	ca	0.1005
4	oldpeak	0.0954
5	thal	0.0930
6	age	0.0825
7	chol	0.0781

선택된 특성들은 심장병 예측과 임상적으로도 관련성이 높은 변수들이다. 예를 들어 cp는 흉통 유형, thalach는 최대 심박수, ca는 주요 혈관 수, oldpeak는 운동 후 ST depression을 의미하므로 심혈관 상태와 직접적인 관련이 있다. 다만 본 프로젝트의 데이터는 중복 제거 후 302개로 크기가 제한적이므로, 선택된 특성만으로 최종 모델을 다시 제한하면 정보 손실이 발생할 수 있다고 판단하였다. 따라서 특성 선택 결과는 변수 중요도 해석과 모델 이해를 위한 보조 근거로 활용하였고, 최종 모델 학습에는 전체 13개 입력 변수를 사용하였다.

모델 학습 단계에서는 서로 다른 특성을 가진 세 가지 모델을 비교하였다. 사용한 모델은 Logistic Regression, Support Vector Classifier, Random Forest Classifier이다. Logistic Regression은 기본적인 선형 분류 모델로, 해석이 비교적 쉽고 작은 데이터셋에서도 안정적인 성능을 기대할 수 있다. SVC는 결정 경계를 찾아 분류하는 모델로, 데이터의 경

계가 복잡할 때 유용할 수 있다. Random Forest는 여러 개의 결정트리를 결합한 앙상블 모델로, 비선형 관계와 변수 간 상호작용을 반영할 수 있다.

평가 지표는 balanced accuracy, precision, recall, F1-score, confusion matrix를 사용하였다. 단순 accuracy만 사용하지 않은 이유는 의료 예측 문제에서는 클래스별 예측 성능을 균형 있게 확인해야 하기 때문이다. 특히 심장병 예측에서는 실제 심장병 환자를 정상으로 잘못 예측하는 False Negative가 위험할 수 있으므로 recall을 중요한 지표로 함께 고려하였다.

기본 모델 비교 결과는 다음과 같다.

모델	Balanced Accuracy	Precision	Recall	F1-score
Logistic Regression	0.7992	0.8000	0.8485	0.8235
SVC	0.7841	0.7941	0.8182	0.8060
Random Forest	0.7359	0.7576	0.7576	0.7576

기본 모델 비교 결과, Logistic Regression이 balanced accuracy, recall, F1-score 측면에서 가장 좋은 성능을 보였다. SVC도 비슷한 수준의 성능을 보였으나, Logistic Regression보다 recall과 F1-score가 낮았다. Random Forest는 예상과 달리 상대적으로 낮은 성능을 보였는데, 이는 중복 제거 후 데이터 수가 302개로 줄어들어 복잡한 모델이 충분히 일반화하기 어려웠기 때문으로 해석할 수 있다.

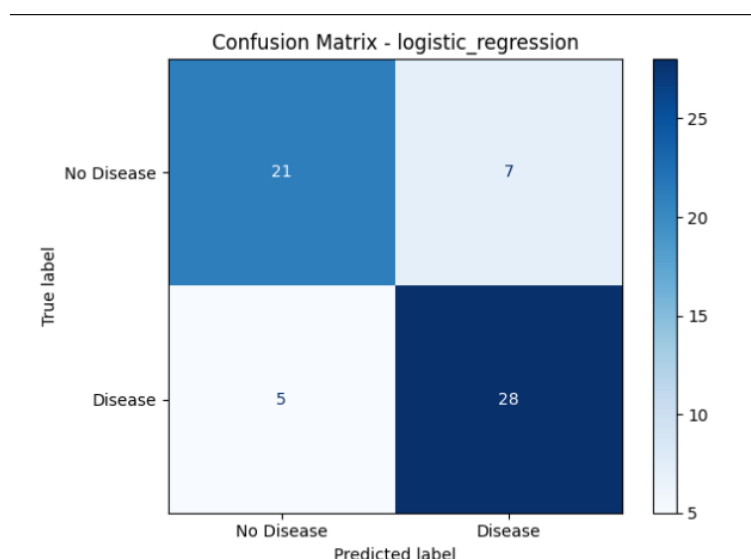


그림 3. Logistic Regression의 confusion matrix

모델 학습 과정은 MLflow를 사용하여 기록하였다. MLflow에는 각 모델의 하이퍼파라미터, balanced accuracy, precision, recall, F1-score, 5-fold cross validation 결과, confusion

matrix 이미지, 모델 artifact를 기록하였다. 이를 통해 여러 실험 결과를 한 화면에서 비교하고, 최종 모델 선택 근거를 체계적으로 관리할 수 있었다.

☐	Run Name	Created ↕	Dataset	Duration	Source	Models
☐	● tuned_logistic_regression	✓ 7 days ago	-	8.9s	📄 train.py	🔗 model
☐	● random_forest	✓ 7 days ago	-	6.3s	📄 train.py	🔗 model
☐	● svc	✓ 7 days ago	-	5.8s	📄 train.py	🔗 model
☐	● logistic_regression	✓ 7 days ago	-	7.9s	📄 train.py	🔗 model

그림 4 - MLflow runs

또한 모델 비교의 안정성을 높이기 위해 5-fold cross validation을 수행하였고, 가장 성능이 좋았던 Logistic Regression에 대해서는 GridSearchCV를 사용하여 하이퍼파라미터 C 값을 탐색하였다. MLflow에는 각 모델의 파라미터, 평가 지표, confusion matrix, 모델 artifact뿐 아니라 모델 계열 정보를 함께 기록하여 실험 결과를 재현 가능하게 관리하였다.

5. 최종 모델 선택 이유

기본 모델 비교 이후 Logistic Regression에 대해 하이퍼파라미터 튜닝을 수행하였다. 튜닝은 GridSearchCV를 사용하였고, Logistic Regression의 규제 강도를 조절하는 C 값을 대상으로 탐색하였다. 또한 5-fold cross validation을 사용하여 모델의 안정적인 성능을 확인하였다. 튜닝 결과 최적의 C 값은 0.1로 선택되었다.

튜닝된 Logistic Regression의 테스트 성능은 다음과 같다.

모델	Best Parameter	Balanced Accuracy	Precision	Recall	F1- score
Tuned Logistic Regression	C=0.1	0.8144	0.8056	0.8788	0.8406

튜닝된 Logistic Regression은 기본 Logistic Regression보다 balanced accuracy, recall, F1-score가 모두 향상되었다. 특히 recall은 기본 Logistic Regression의 0.8485에서 0.8788로 증가하였다. 심장병 예측 문제에서는 실제 심장병 환자를 정상으로 잘못 예측하는 False Negative가 특히 위험하다. False Negative가 발생하면 환자가 필요한 추가 검사나 진료를 받지 못할 수 있기 때문이다. 따라서 본 프로젝트에서는 단순 accuracy보다 recall과 balanced accuracy를 중요한 기준으로 고려하였다.

최종 모델로 tuned Logistic Regression을 선택한 이유는 다음과 같다. 첫째, 테스트셋에서 가장 높은 balanced accuracy를 보였다. 둘째, 의료 예측에서 중요한 recall이 0.8788로 가장 높게 나타났다. 셋째, F1-score도 0.8406으로 precision과 recall 사이의 균형이

가장 좋았다. 넷째, Logistic Regression은 Random Forest와 같은 복잡한 모델에 비해 해석 가능성이 상대적으로 높기 때문에 의료 보조 모델로 설명하기에 적합하다.

따라서 본 프로젝트에서는 성능, recall, 해석 가능성을 종합적으로 고려하여 tuned Logistic Regression을 최종 모델로 선택하였다. 최종 모델은 models/best_tuned_model.joblib 파일로 저장하였다.

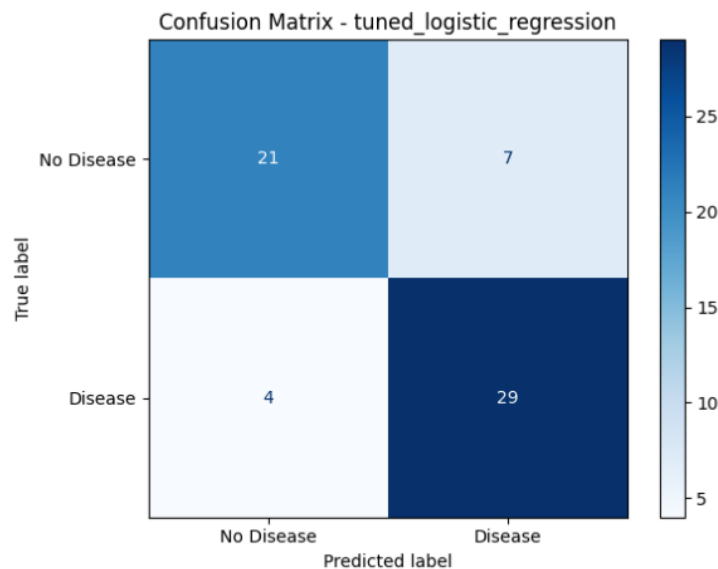


그림 5 - confusion matrix tuned logistic regression

6. 테스트와 Docker 패키징

모델 학습 이후에는 저장된 모델이 실제로 안정적으로 동작하는지 확인하기 위해 unittest 기반 단위 테스트를 작성하였다. 테스트 파일은 tests/test_pipeline.py에 작성하였으며, 최종 모델과 추론 파이프라인이 정상적으로 동작하는지 확인하는 데 초점을 두었다.

테스트 항목은 총 6개이다. 첫째, 예측 결과의 행 개수가 입력 데이터의 행 개수와 일치하는지 확인하였다. 둘째, 예측 확률이 0 이상 1 이하 범위에 있는지 확인하였다. 셋째, 각 샘플에 대한 클래스별 예측 확률의 합이 1에 가까운지 확인하였다. 넷째, 정상적인 입력 데이터가 검증 함수를 통과하는지 확인하였다. 다섯째, 비정상적인 chol 값이 입력되었을 때 ValueError가 발생하는지 확인하였다. 여섯째, 동일한 입력에 대해 동일한 예측 결과가 나오는지 확인하였다. 이러한 테스트는 모델이 입력 데이터에 대해 올바른 형태의 예측값을 반환하는지, 확률 출력이 정상적인 분류 확률 형태인지, 임상적으로 비정상적인 입력값이 모델에 그대로 전달되지 않는지, 그리고 동일한 입력에 대해 재현 가능한 결과를 제공하는지를 검증하기 위한 것이다. 로컬 환경에서 `python -m unittest`

tests/test_pipeline.py 명령어를 실행한 결과 6개의 테스트가 모두 통과하였다.

```
샘플 입력 파일 생성 완료: data/sample_input.csv
.....
-----
Ran 6 tests in 6.699s
OK
```

그림 6. unittest 테스트 통과 결과

또한 GitHub Actions를 사용하여 CI 워크플로를 구성하였다. main 브랜치에 push가 발생하면 GitHub Actions에서 Ubuntu 환경을 구성하고, Python 3.11을 설치한 뒤, requirements.txt를 기반으로 필요한 라이브러리를 설치하고, unittest를 자동 실행하도록 하였다. 초기 CI 실행에서는 models/best_tuned_model.joblib 파일이 GitHub 저장소에 포함되지 않아 FileNotFoundError가 발생하였다. 이후 모델 파일을 저장소에 추가한 뒤 다시 push하였고, 최종적으로 GitHub Actions CI가 초록색 체크로 통과하였다.

✔ Update README with final project results
CardioCare CI #3: Commit [3126592](#) pushed by [wuyung027](#)

main

그림 7. GitHub Actions CI 통과 결과

Docker 패키징도 수행하였다. Dockerfile은 Python 3.11 slim 이미지를 기반으로 작성하였고, requirements.txt를 통해 필요한 라이브러리를 설치한 뒤, 컨테이너 실행 시 src/inference.py가 실행되도록 구성하였다. Docker 빌드 과정에서 처음에는 requirements.txt에 Windows 전용 패키지인 pywin32가 포함되어 Linux 기반 Docker 컨테이너에서 설치 오류가 발생하였다. 이 문제는 기존 pip freeze 기반 requirements 파일에 불필요한 패키지가 많이 포함되어 있었기 때문에 발생하였다. 이후 프로젝트 실행에 필요한 핵심 라이브러리만 남기도록 requirements.txt를 정리하여 Docker 빌드를 성공시켰다. Docker 실행 명령어는 다음과 같다.

```
docker build -t cardiocare:1.0 .
```

```
docker run cardiocare:1.0
```

Docker 실행 결과, 저장된 최종 모델이 샘플 입력 데이터에 대해 정상적으로 예측을 수행하는 것을 확인하였다.

```

예측 결과
=====

샘플 1
예측값: 0 (정상 가능성 높음)
심장병 예측 확률: 0.3030

샘플 2
예측값: 1 (심장병 가능성 있음)
심장병 예측 확률: 0.8549
예측 결과 저장 완료: reports/predictions.csv

```

그림 8. Docker 컨테이너 기반 추론 실행 결과

7. 모니터링 및 Drift Detection

모델이 실제 운영 환경에서 사용될 경우, 시간이 지나면서 입력 데이터의 분포가 학습 시점과 달라질 수 있다. 예를 들어 환자 집단의 특성, 측정 장비, 병원 환경, 데이터 수집 방식이 바뀌면 모델이 학습한 데이터 분포와 실제 입력 데이터 분포 사이에 차이가 발생할 수 있다. 이러한 현상을 data drift라고 하며, drift가 발생하면 모델 성능이 저하될 수 있다. 본 프로젝트에서는 src/monitor.py를 작성하여 추론 로그 기록과 drift detection을 수행하였다. 추론 과정에서는 logging을 사용하여 timestamp, model version, input shape, prediction, actual label을 logs/inference_monitor.log 파일에 기록하였다. 이를 통해 모델이 언제 어떤 입력에 대해 어떤 예측을 수행했는지 추적할 수 있도록 하였다.

Drift 상황을 실험하기 위해 테스트 데이터의 일부 연속형 변수 값을 인위적으로 이동시켰다. 구체적으로 chol, trestbps, oldpeak 값을 증가시켜 학습 데이터와 다른 분포를 가진 drift 데이터를 생성하였다. 이후 학습 데이터 분포와 drift 데이터 분포를 비교하기 위해 연속형 변수인 age, trestbps, chol, thalach, oldpeak에 대해 Kolmogorov-Smirnov test를 수행하였다. p-value가 0.05보다 작은 경우 해당 변수에서 drift가 발생한 것으로 판단하였다. KS-test 결과, trestbps, chol, oldpeak에서 p-value가 0.05보다 작게 나타나 drift가 탐지되었다. 반면 age와 thalach는 p-value가 0.05보다 크게 나타나 drift가 탐지되지 않았다. 이는 인위적으로 값을 이동시킨 변수에서 실제로 분포 변화가 탐지되었음을 보여준다. Drift 발생 전후의 성능도 비교하였다. Original test data에서는 balanced accuracy 0.8144, recall 0.8788, F1-score 0.8406을 기록하였다. 그러나 drift data에서는 balanced accuracy가 0.7646, recall이 0.6364, F1-score가 0.7368로 감소하였다. 특히 recall이 크게 감소한 것은 의료 예측 모델에서 중요한 문제이다. 심장병 예측에서는 실제 심장병 환자를 놓치지 않는 것이 중요하므로, drift로 인한 recall 감소는 운영 환경에서 반드시 모니터링해야 한다.

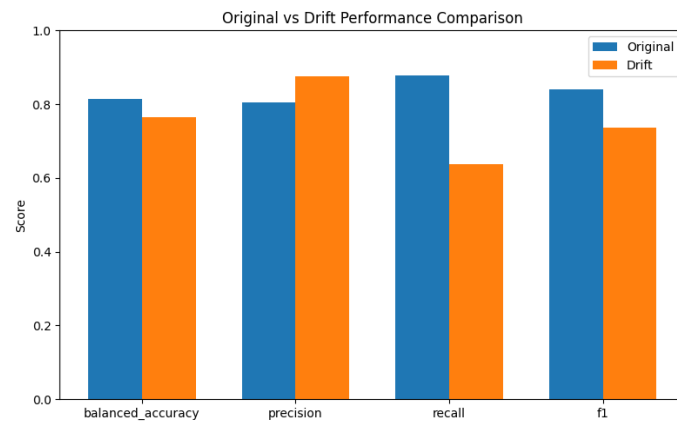


그림 9. Original data와 drift data의 성능 비교

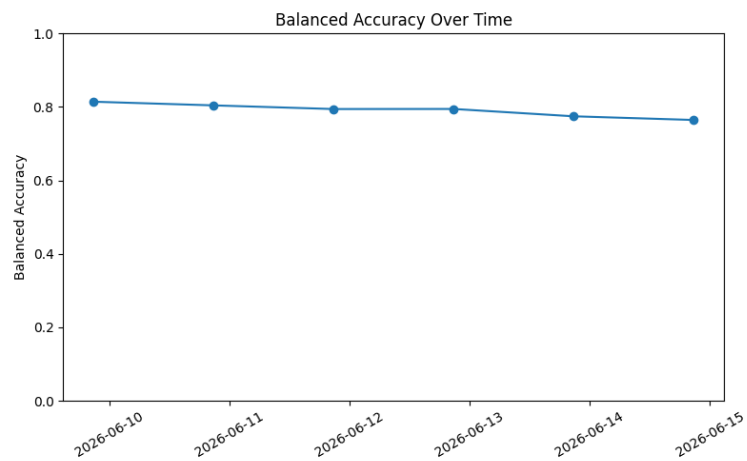


그림 10. 시간 흐름에 따른 balanced accuracy 변화 예시

이러한 모니터링 결과를 바탕으로 운영 환경에서는 drift가 탐지되거나 balanced accuracy, recall과 같은 주요 지표가 일정 기준 이하로 하락할 경우 재학습을 검토할 수 있다. 특히 의료 예측 모델에서는 성능 저하가 환자 안전에 영향을 줄 수 있으므로, 자동 재학습보다는 사람이 검토한 뒤 재학습 여부를 결정하는 Human-in-the-loop 방식이 필요하다.

8. 재학습 / 피드백 루프 전략

본 프로젝트에서는 데이터 드리프트 실험을 통해 모델 운영 이후 재학습 전략의 필요성을 확인하였다. 실제 서비스 환경에서는 새로운 환자 데이터가 계속 들어올 수 있으며, 시간이 지나면서 환자군의 특성이나 측정 환경이 달라질 수 있다. 따라서 모델의 성능을 안정적으로 유지하기 위해서는 주기적인 모니터링과 재학습 구조가 필요하다.

재학습 전략은 다음과 같이 설계할 수 있다. 먼저 운영 중 들어오는 입력 데이터의 주요 변수 분포를 기록한다. 예를 들어 chol, trestbps, thalach, oldpeak와 같은 주요 임상 변수의 평균, 표준편차, 분위수 등을 주기적으로 저장한다. 이후 학습 시점의 기준 데이터와 최근 입력 데이터의 분포를 비교한다. 이때 KS 검정이나 PSI와 같은 통계적 방법을 사용할 수 있다.

드리프트가 감지되면 모델 성능 지표를 함께 확인한다. 특히 본 프로젝트처럼 의료 예측 문제에서는 recall, balanced accuracy, confusion matrix를 중요하게 확인해야 한다. 만약 recall이 일정 기준 이하로 떨어지거나, 입력 변수에서 유의미한 분포 변화가 반복적으로 감지된다면 재학습을 수행한다.

재학습 과정에서는 새로 수집된 데이터와 기존 데이터를 함께 사용하여 모델을 다시 학습한다. 이후 기존 모델과 새 모델의 성능을 비교하고, 새 모델이 recall과 balanced accuracy 측면에서 더 안정적인 성능을 보일 경우에만 교체한다. 이때 MLflow를 활용하면 기존 모델과 새 모델의 성능, 하이퍼파라미터, artifact를 함께 관리할 수 있다.

피드백 루프는 새 환자 데이터가 입력되면 모델이 예측을 수행하고, 입력 데이터와 예측 결과를 로그로 저장하는 방식으로 구성할 수 있다. 이후 실제 정답 또는 후속 진단 결과가 확보되면 예측 결과와 실제 결과를 비교하여 성능 저하 여부를 확인한다. 만약 주요 변수에서 데이터 드리프트가 반복적으로 감지되거나 recall, balanced accuracy와 같은 핵심 지표가 기준 이하로 하락하면 재학습 데이터셋을 구성하고 모델을 다시 학습한다. 이후 기존 모델과 새 모델의 성능을 비교하여 새 모델이 더 안정적인 성능을 보일 경우에만 모델을 교체한다.

이러한 피드백 루프를 통해 모델이 배포 이후에도 지속적으로 개선될 수 있도록 설계할 수 있다. 다만 의료 예측 모델에서는 드리프트가 탐지되었다고 해서 자동으로 재학습 후 즉시 배포하는 방식은 적절하지 않다. 새로 수집된 데이터에 오류가 있거나 특정 환자군의 데이터가 과도하게 반영될 경우, 모델이 잘못된 방향으로 반복 학습되는 runaway feedback 문제가 발생할 수 있기 때문이다. 따라서 본 프로젝트에서는 Human-in-the-loop 방식을 적용하여, 드리프트 탐지와 성능 저하가 확인되면 개발자 또는 의료 전문가가 데이터 품질, 라벨 정확성, 성능 변화 원인을 검토한 뒤 재학습 및 모델 교체 여부를 결정하는 구조가 필요하다고 판단하였다.

9. 서빙 방식 선택

본 프로젝트에서는 최종 모델을 `models/best_tuned_model.joblib` 파일로 저장하고, `src/inference.py`를 통해 저장된 모델을 불러와 예측을 수행하는 배치 추론 방식을 구현하였다. 또한 Docker를 사용하여 동일한 실행 환경에서 모델을 실행할 수 있도록 패키징하였다. 현재 구현 단계에서는 `docker run cardiocare:1.0` 명령어를 통해 저장된 모델을 불러오고, 샘플 입력 데이터에 대한 예측 결과와 심장병 예측 확률을 출력하도록 구성하였다.

이번 프로젝트에서 구현한 기본 실행 방식은 Docker 기반 배치 추론이다. 이 방식을 선택한 이유는 과제 수준에서 실시간 API 서버를 구축하는 것보다, 모델이 독립된 환경에서도 재현 가능하게 실행되는지를 확인하는 것이 더 중요하다고 판단했기 때문이다. Docker를 사용하면 교수자나 다른 사용자가 동일한 라이브러리와 실행 환경에서 모델을 실행할 수 있으며, 로컬 환경 차이로 인한 오류를 줄일 수 있다.

향후 실제 서비스로 확장한다면 MaaS(Model as a Service) 방식이 적합하다고 판단하였다. MaaS 방식은 모델을 서버 또는 API 형태로 배포하고, 사용자가 입력한 환자 정보를 서버에 전달하면 서버에서 예측 결과를 반환하는 구조이다. 본 프로젝트의 심장병 예측 모델은 실시간 응급 처치 판단보다는 진료 보조를 위한 위험도 예측에 가깝기 때문에, 매우 낮은 지연시간이 필요한 On-Device 방식보다는 서버 기반 MaaS 방식으로도 충분히 활용 가능하다고 판단하였다.

MaaS 방식을 선택하는 또 다른 이유는 모델 업데이트와 유지보수가 용이하기 때문이다. 의료 데이터는 시간이 지나면서 환자군의 특성이나 측정 환경이 달라질 수 있으므로, 드리프트가 발생하면 모델을 재학습하고 새로운 버전으로 교체해야 한다. MaaS 방식에서는 서버에 배포된 모델만 교체하면 되므로, 여러 사용자 기기에 각각 모델을 다시 배포해야 하는 On-Device 방식보다 모델 관리가 쉽다.

또한 MaaS 방식은 모니터링과 로그 관리에 유리하다. 서버에서 추론 요청, 입력 데이터 형태, 예측 결과, 실제 라벨 여부, 모델 버전 등을 중앙에서 기록할 수 있기 때문에 drift detection과 성능 저하 감지에 적합하다. 본 프로젝트에서 작성한 `src/monitor.py`의 추론 로그 기록과 drift detection 구조도 향후 MaaS 방식으로 확장하기 쉽다.

다만 의료 데이터는 PHI와 같은 민감한 개인정보를 포함할 수 있으므로, MaaS 방식에서는 데이터 전송과 저장 과정의 보안 관리가 중요하다. 실제 운영 환경에서는 개인정보 비식별화, 암호화, 접근 권한 관리, 로그 보관 정책이 함께 적용되어야 한다. On-Device 방식은 환자 데이터가 기기 밖으로 나가지 않는다는 장점이 있지만, 본 프로젝트에서는 모델 업데이트, 모니터링, 재학습 관리가 중요하므로 MaaS 방식이 더 적합하다고 판단하였다.

현재 단계에서는 Docker 기반 배치 추론을 통해 모델이 재현 가능하게 동작하는 것을 우선적으로 구현하였다. 이후 확장 방향으로 FastAPI 기반 REST API 서버, 로그 저장 DB 연동, 모델 버전 관리, 모니터링 대시보드 구축 등을 고려할 수 있다.

10. 한계, 윤리적 고려 사항 및 AI 도구 사용 공개

본 프로젝트는 공개 데이터셋을 기반으로 한 학습용 머신러닝 프로젝트이다. 따라서 실제 의료 현장에 바로 적용하기에는 여러 한계가 존재한다.

첫째, 데이터셋의 규모가 제한적이다. 원본 데이터는 1025개 행이었으나 중복 제거 후 실제 학습과 평가에 사용한 데이터는 302개 샘플이다. 의료 데이터는 환자군의 다양성이 매우 중요하므로, 더 큰 규모의 실제 임상 데이터가 필요하다.

둘째, 공개 데이터셋은 특정 환경에서 수집된 데이터이기 때문에 실제 병원 환경의 다양한 환자 특성을 충분히 반영하지 못할 수 있다. 예를 들어 연령대, 성별, 질환 이력, 측정 장비, 지역적 차이 등이 실제 환경에서는 더 복잡하게 작용할 수 있다.

셋째, 모델의 예측 결과는 의학적 진단을 대체할 수 없다. 본 프로젝트의 모델은 심장병 가능성을 예측하는 보조 도구로만 사용되어야 하며, 최종 진단과 치료 결정은 반드시 전문 의료진이 수행해야 한다.

넷째, False Negative 문제에 주의해야 한다. 실제 심장병 환자를 정상으로 예측하면 환자가 필요한 진료를 받지 못할 위험이 있다.

따라서 본 프로젝트에서는 accuracy만 보지 않고 recall, balanced accuracy, confusion matrix를 함께 확인하였다.

다섯째, 데이터 드리프트에 취약할 수 있다. 본 프로젝트의 드리프트 실험에서 chol과 trestbps 값이 변화했을 때 recall이 0.8788에서 0.6364로 감소하였다. 이는 실제 운영 환경에서 데이터 분포 변화가 모델 성능에 영향을 줄 수 있음을 보여준다.

윤리적으로는 모델의 예측 결과를 과신하지 않는 것이 중요하다. 의료 분야에서 AI 모델은 판단을 보조하는 도구일 뿐, 최종 결정을 내리는 주체가 되어서는 안 된다. 또한 모델 사용 시 데이터 개인정보 보호, 편향 가능성, 모델 설명 가능성, 지속적인 성능 검증이 함께 고려되어야 한다.

본 프로젝트를 진행하는 과정에서 AI 도구를 일부 활용하였다. AI 도구는 프로젝트 방향을 대신 결정하기 위한 용도가 아니라, 구현 과정에서 발생한 오류를 분석하고, 코드 구조를 점검하며, 보고서 문장을 정리하는 보조 도구로 사용하였다.

AI 도구를 활용한 주요 부분은 첫째, 프로젝트 구조 설계 과정에서 도움을 받았다. 데이터 로딩, EDA, 전처리, 모델 학습, 추론, 테스트, Docker, CI, 모니터링으로 이어지는 End-

to-End 머신러닝 프로젝트 구조를 구성하는 데 AI 도구를 참고하였다.

둘째, 오류 메시지 분석에 활용하였다. MLflow UI에서 실험 기록이 Default만 표시되는 문제가 발생했을 때, AI 도구를 활용하여 원인을 점검하였다. 그 결과 MLflow의 tracking URI 설정 문제와 PowerShell 터미널 환경 문제 가능성을 확인하였다. 이후 `mlflow.set_tracking_uri("file:./mlruns")`를 명시하고, PowerShell 대신 Command Prompt에서 재실행하여 CardioCare_Heart_Disease 실험 기록을 정상적으로 확인하였다.

셋째, Docker 빌드 오류 해결에 활용하였다. Docker 이미지 빌드 과정에서 requirements.txt에 Windows 전용 패키지인 pywin32가 포함되어 Linux 기반 Docker 컨테이너에서 설치 오류가 발생하였다. AI 도구를 활용하여 오류 원인을 확인한 뒤, 프로젝트 실행에 필요한 핵심 라이브러리만 남기도록 requirements.txt를 정리하였다. 이후 Docker 빌드와 실행이 정상적으로 완료되었다.

넷째, 보고서 및 README 작성 보조에 활용하였다. 모델 성능 결과, 드리프트 실험 결과, GitHub Actions CI 결과 등을 보고서 형식에 맞게 정리하는 과정에서 AI 도구의 도움을 받았다.

다만 최종 코드 실행, 결과 확인, 모델 성능 검증, Docker 실행, GitHub Actions 확인은 직접 수행하였다. AI 도구는 프로젝트 수행을 보조하는 역할로 사용하였으며, 최종 결과물에 대한 검토와 책임은 작성자에게 있다.